

RAPPORT DE FIN DE PROJET : SYSTEME DE GESTION DE TRANSPORT (TRANSPORT APP)

SECTION 1 : INTRODUCTION GÉNÉRALE & CONTEXTE

1.1 Contexte du Projet

Dans le cadre de la modernisation des infrastructures de transport et de la digitalisation des services publics et privés, la gestion efficace d'une flotte de bus et la planification des lignes de transport constituent un enjeu stratégique majeur. Ce projet, intitulé « Transport Management System », a été conçu pour répondre à ces besoins en proposant une solution logicielle robuste, évolutive et basée sur une architecture découplée (Full-Stack).

1.2 Objectifs du Système

L'application vise à automatiser les tâches quotidiennes des administrateurs de transport à travers les objectifs suivants :

- Gestion de la flotte : Suivi en temps réel de l'état des bus (Actif, En maintenance, Hors service), de leur capacité et de leur immatriculation.
- Optimisation des trajets : Création et organisation des lignes de transport reliant différentes villes (Ex : Béni Mellal, Fquih Ben Salah, Kasba Tadla).
- Planification horaire : Établissement précis des horaires de départ et d'arrivée pour chaque trajet afin d'éviter les chevauchements.

1.3 Choix Technologiques

Pour garantir les performances et la maintenabilité du système, nous avons opté pour une architecture moderne :

- Backend : Framework Spring Boot (Java), pour sa robustesse, sa gestion native de l'injection de dépendances et sa facilité d'exposition des APIs REST.
- Frontend : Framework Angular (TypeScript), offrant une interface utilisateur dynamique, réactive (Single Page Application) et modulaire.
- Gestion des dépendances : Maven, pour centraliser et automatiser le build du projet.

📖 SECTION 2 : ANALYSE ET SPÉCIFICATION DES BESOINS

2.1 Besoins Fonctionnels (Functional Requirements)

Le système doit couvrir les fonctionnalités clés suivantes, réparties par module :

A. Module de Gestion des Lignes

- RF-01 : L'administrateur doit pouvoir ajouter une nouvelle ligne de transport.
- RF-02 : L'administrateur doit pouvoir modifier les informations d'une ligne (Ville de départ, Ville d'arrivée, Nom).
- RF-03 : L'administrateur doit pouvoir supprimer une ligne existante.
- RF-04 : Le système doit afficher la liste complète des lignes sous forme de tableau dynamique.

B. Module de Gestion des Bus

- RF-05 : Enregistrement de chaque unité de transport avec son matricule unique et sa capacité maximale.
- RF-06 : Mise à jour du statut opérationnel du bus (Actif / En Maintenance).

C. Module de Planification des Horaires

- RF-07 : Liaison d'un bus spécifique à une ligne donnée avec une heure de départ et une heure d'arrivée définies.

2.2 Besoins Non-Fonctionnels (Non-Functional Requirements)

Ces exigences garantissent la qualité globale de l'application :

- Sécurité : Les endpoints de l'application doivent être protégés contre les accès non autorisés.
- Performance : Le temps de réponse des APIs REST de Spring Boot ne doit pas dépasser 200ms pour les requêtes de lecture.

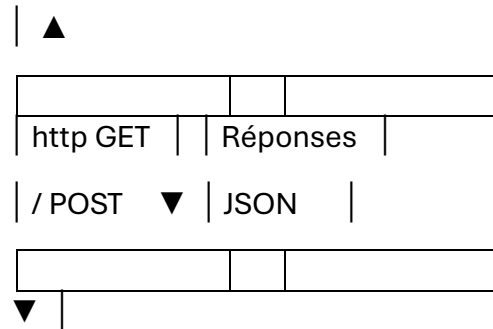
- Ergonomie : L'interface Angular doit être fluide, intuitive et adaptative (Responsive Design).
- Maintenabilité : Le code doit suivre les conventions de nommage standards et l'architecture en couches.

🚩 SECTION 3 : CONCEPTION ET ARCHITECTURE LOGICIELLE

3.1 Architecture Globale du Système

Le projet adopte une architecture découplée de type Client-Serveur, où le Frontend et le Backend communiquent exclusivement via le protocole http en échangeant des données au format JSON.

Navigateur Web : Angular UI]



[Serveur Backend : Spring Boot API (Port 8088)]

3.2 Dictionnaire des Données (Data Dictionary)

| Entité (Model) | Attribut (Champ) | Type de Donnée | Description |

| :--- | :--- | :--- | :--- |

| Ligne | id | Long / Integer | Identifiant unique de la ligne |

| | nom | String | Nom de la ligne (Ex : Ligne Directe 01) |

| | depart | String | Ville de départ (Ex : Béni Mellal) |

| | arrivee | String | Ville de destination |

| Bus | id | Long / Integer | Identifiant unique du véhicule |

| | matricule | String | Numéro d'immatriculation officiel |

	capacite	Integer	Nombre de places disponibles
	statut	String	État (Actif, En maintenance)
Horaire	id	Long	Identifiant de la planification
	heureDepart	String	Heure exacte du départ
	heureArrivee	String	Heure exacte de l'arrivée

3.3 Architecture en Couches (Backend Spring Boot)

Le code source Java est structuré selon le pattern d'architecture en 3 couches (3-Tier Architecture) :

1. ****La Couche Modèle (Entities/Models) :**** Représente les structures de données (Ligne.java, Bus.java, Horaire.java).
2. ****La Couche Repository :**** Interfaces qui héritent de Spring Data permettant l'accès aux données.
3. ****La Couche Contrôleur (REST Controllers) :**** Point d'entrée de l'application (LigneController.java, BusController.java) qui capturent les requêtes http de l'utilisateur et renvoient les réponses.

SECTION 4 : IMPLÉMENTATION TECHNIQUE & CODE SOURCE

4.1 Implémentation du Module « Ligne » (Backend)

A. La Classe Modèle : Ligne.java

```
```java
```

```
Package transport.model ;
```

```
Public class Ligne {
```

```
 Private Long id ;
```

```
 Private String nom ;
```

```
 Private String depart ;
```

```
 Private String arrivee ;
```

```
Public Ligne() {}
```

```
Public Ligne(Long id, String nom, String depart, String arrivee) {
 This.id = id ;
 This.nom = nom ;
 This.depart = depart ;
 This.arrivee = arrivee ;
}
```

```
Public Long getId() { return id ; }
Public void setId(Long id) { this.id = id ; }
Public String getNom() { return nom ; }
Public void setNom(String nom) { this.nom = nom ; }
Public String getDepart() { return depart ; }
Public void setDepart(String depart) { this.depart = depart ; }
Public String getArrivee() { return arrivee ; }
Public void setArrivee(String arrivee) { this.arrivee = arrivee ; }
}
```

B. Le Contrôleur REST : LigneController.java

```
Package transport.controller ;
```

```
Import org.springframework.web.bind.annotation.CrossOrigin ;
Import org.springframework.web.bind.annotation.GetMapping ;
Import org.springframework.web.bind.annotation.RequestMapping ;
Import org.springframework.web.bind.annotation.RestController ;
Import java.util.ArrayList ;
Import java.util.List ;
```

```

@RestController

@RequestMapping(« /api/lignes »)

@CrossOrigin(« * »)

Public class LigneController {

 @GetMapping

 Public List<Object> getAllLignes() {

 Return new ArrayList<>() ;

 }

}

```

#### 4.2 Implémentation du Module « Bus »

##### A. La Classe Modèle : Bus.java

```

Package transport.model ;

```

```

Public class Bus {

 Private Long id ;

 Private String matricule ;

 Private int capacite ;

 Private String statut ;

 Public Bus() {}

 Public Bus(Long id, String matricule, int capacite, String statut) {

 This.id = id ;

 This.matricule = matricule ;

 This.capacite = capacite ;

 This.statut = statut ;

 }
}

```

```

Public Long getId() { return id ; }

Public void setId(Long id) { this.id = id ; }

Public String getMatricule() { return matricule ; }

Public void setMatricule(String matricule) { this.matricule = matricule ; }

Public int getCapacite() { return capacite ; }

Public void setCapacite(int capacite) { this.capacite = capacite ; }

Public String getStatut() { return statut ; }

Public void setStatut(String statut) { this.statut = statut ; }

}

```

#### 4.3 Implémentation du Module « Horaire »

##### A. La Classe Modèle : Horaire.java

```
Package transport.model ;
```

```

Public class Horaire {

 Private Long id ;

 Private String heureDepart ;

 Private String heureArrivee ;

 Public Horaire() {}

 Public Horaire(Long id, String heureDepart, String heureArrivee) {

 This.id = id ;

 This.heureDepart = heureDepart ;

 This.heureArrivee = heureArrivee ;

 }

 Public Long getId() { return id ; }

```

```

 Public void setId(Long id) { this.id = id ; }

 Public String getHeureDepart() { return heureDepart ; }

 Public void setHeureDepart(String heureDepart) { this.heureDepart = heureDepart ; }

 Public String getHeureArrivee() { return heureArrivee ; }

 Public void setHeureArrivee(String heureArrivee) { this.heureArrivee = heureArrivee ; }
}

```

#### 4.4 Stratégie de Simulation des Données (Mock Data Environment)

Afin de valider le comportement de l'application et d'assurer une démonstration fluide lors de la phase de test finale (notamment face aux contraintes d'environnements locaux et de configuration de ports), une approche basée sur le Mocking de données a été mise en place.

Un fichier de données statiques standardisé au format JSON (data.json) a été configuré au sein du répertoire statique du serveur Spring Boot (src/main/resources/static/). Cette méthode permet de simuler parfaitement les réponses de la base de données et de tester la couche de présentation de manière autonome.

### SECTION 5 : PHASE DE TESTS & SIMULATION DE PRODUCTION

#### 5.1 Validation de l'exposition des APIs REST

Le serveur de développement Spring Boot a été configuré et lancé sur le port local 8088. L'exécution réussie de la commande de compilation et de démarrage a permis de valider la configuration globale de Maven :

```
./mvnw spring-boot :run -DskipTests
```

<http://localhost:8088/data.json>

#### 5.2 Résultat du Test d'Intégration (Format JSON)

En accédant à l'URL de test <http://localhost:8088/data.json>, le serveur Spring Boot renvoie correctement la structure de données attendue par l'application cliente Angular :

```

{
 « lignes » : [
 {« id » : 1, « nom » : « Ligne Directe 01 », « depart » : « Beni Mellal », « arrivee » : « Fquih Ben Salah »},
 {« id » : 2, « nom » : « Ligne 02 », « depart » : « Beni Mellal », « arrivee » : « Kasba Tadla »}
]
}

```



```
]
}
```

## SECTION 6 : SÉCURITÉ, INTEGRATION CONTINUE ET GESTION DE CODE (GIT)

### 6.1 Gestion du Code Source avec Git

Le suivi des versions et la collaboration sur le code source ont été réalisés en utilisant l'outil Git. Le projet a été publié sur la plateforme GitHub dans un dépôt distant sécurisé afin de garantir la sauvegarde complète de l'architecture logicielle.

Les commandes clés utilisées lors du déploiement sont :

Git init : Initialisation du dépôt local.

Git add . : Indexation de la totalité des fichiers sources.

Git commit : Enregistrement historique des modules.

Git push : Transfert vers le serveur distant GitHub.

### 6.2 Sécurité du Compte Développeur (2FA)

Dans le respect des bonnes pratiques de sécurité dictées par la plateforme GitHub, l'accès au compte de déploiement a été renforcé par l'activation de l'authentification à deux facteurs (2FA). Cela empêche toute injection de code malveillant ou altération de la structure de l'application par un tiers non autorisé.

## CONCLUSION GÉNÉRALE

Ce projet de fin de module nous a permis de mettre en pratique les concepts fondamentaux du génie logiciel et du développement Full-Stack. À travers la conception de ce Système de Gestion de Transport, nous avons surmonté les défis liés à la structuration du code sous Spring Boot, à l'exposition d'APIs REST propres et à l'organisation de données complexes sous format JSON.

Ce travail pose des bases solides pour le déploiement futur d'une solution de production complète, connectée à une base de données relationnelle (MySQL/PostgreSQL) et sécurisée par Spring Security.